

TOP 10 MOST ASKED LLD DESIGN PATTERNS

≈ Every SDE Must Know ≈



Why Design Patterns?

- ✓ Proven solutions to common design problems.
- ✓ Improve code reusability, maintainability and scalability.
- ✓ Reduce complexity and make code easy to understand.
- ✓ Help you think like a senior engineer.



Remember

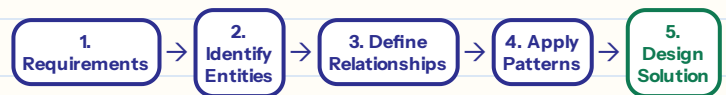
Patterns are not about copy-paste code. They are about solving problems in the best possible way.



How to Use This Guide?

- ★ Understand the problem.
- ★ Identify where the pattern fits.
- ★ Apply pattern and design the solution.
- ★ Keep practicing with real-world examples.

Design Flow (High Level)



• The Top 10 Design Patterns •

1. Singleton

Ensures a class has only one instance and provides a global point of access to it.

Use cases: Logger, Config Manager, Cache, DB Connection Pool

2. Factory

Creates objects without exposing the creation logic to the client.

Use cases: Notification System, Payment Gateway, Vehicle Factory, UI Components

3. Strategy

Defines a family of algorithms, encapsulates each one, and makes them interchangeable.

Use cases: Payment Methods, Sorting Algorithms, Pricing Engine, Discount Calculation

4. Observer

Defines a one-to-many dependency so that when one object changes state, all dependents are notified.

Use cases: Notification Service, Event Bus, Stock Price Updates, Chat Applications

5. Builder

Separates construction of a complex object from its representation.

Use cases: User Builder, SQL Query Builder, HTTP Request Builder, Document Generator

6. Decorator

Dynamically adds behavior/responsibilities to objects without modifying their code.

Use cases: Coffee Machine, Pizza Toppings, Logging, Compression, Encryption

7. Adapter

Allows incompatible interfaces to work together.

Use cases: Third-party APIs, Payment Providers, Legacy Systems, Database Migration

8. Command

Encapsulates a request as an object, letting you parameterize clients with different requests.

Use cases: Undo/Redo, Task Scheduler, Queue Processing, Remote Controls

9. State

Allows an object to alter its behavior when its internal state changes.

Use cases: ATM Machine, Elevator, Vending Machine

10. Chain of Responsibility

Passes a request along a chain of handlers until one of them handles it.

Use cases: Authentication, Logging, Approval Workflow

LLD DESIGN PROCESS

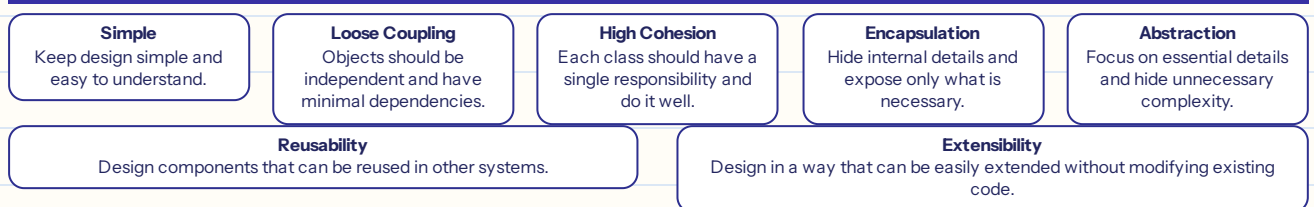
≈ How to Solve any LLD Problem ≈

Page 2 of 10

• The LLD Design Steps •



• Key Principles to Follow •



Choosing Objects — Tips

→ Look for Nouns in the problem statement.

Example: Problem: Design a Parking Lot System.
Nouns: ParkingLot, Vehicle, ParkingSpot, Ticket, Payment, Entrance, Exit.

- ✓ Don't blindly create classes. Think about responsibilities.
- ✓ One class = One responsibility (SRP).
- ✓ Avoid creating God classes.
- ✓ Prefer composition over inheritance.
- ✓ Program to interfaces, not implementations.

Relationship Types

- ★ **Association (uses)** — A → B
- ★ **Aggregation (has-a)** — A ◇→ B
- ★ **Composition (strong has-a)** — A ◆→ B
- ★ **Inheritance (is-a)** — A ▷ B
- ★ **Dependency (depends on)** — A -.-> B

Common Multiplicities

1	→ Exactly one
0..1	→ Zero or one
0..*	→ Zero or more
1..*	→ One or more
m..n	→ Between m and n
n	→ Exactly n

Example: ParkingLot (1) —has—> (*) ParkingSpot

HLD vs LLD

≈ Understand the Difference ≈

Aspect	HLD	LLD
 Goal	Overall system design and architecture.	Detailed design of individual components and classes.
 Focus	System components and their relationships.	Classes, methods, attributes and their interactions.
 Level	System-level (Macro view)	Module-level (Micro view)
 Who Creates	Solution Architect	Developers / Designers
 What	Modules, components, data flow, technologies.	Classes, objects, methods, interfaces, relationships.
 Diagram	Architecture Diagram, Component Diagram, Deployment Diagram.	Class Diagram, Sequence Diagram, Activity Diagram, State Diagram.
 Example	Designing the architecture of BookMyShow.	Designing the Booking, Payment and Seat Management modules.

Key Takeaway

HLD is about the big picture. LLD is about the details.
HLD answers **"WHAT"**. LLD answers **"HOW"**.

Example: Ride Booking App

HLD: User Service, Driver Service, Payment Service, Notification Service, DB, APIs.

LLD: User class, Driver class, Booking class, Payment class, methods, attributes, relations.

• Types of Diagrams in LLD •

1. Class Diagram

Shows classes, attributes, methods and relationships.

2. Sequence Diagram

Shows object interactions in chronological order.

3. Activity Diagram

Shows workflow and business processes.

4. State Diagram

Shows states of an object and transitions between them.

5. Use Case Diagram

Shows functionalities from user's perspective.

6. Component Diagram

Shows system components and their dependencies.

7. Deployment Diagram

Shows physical deployment of components.

8. Package Diagram

Shows organization of packages and dependencies.

9. Communication Diagram

Shows object interactions with messages.

10. Object Diagram

Shows runtime objects and their current state.

Tips for LLD Interview

- ✓ Understand the requirements clearly.
- ✓ Choose the right objects and classes.
- ✓ Use design patterns where required.
- ✓ Handle edge cases and exceptions.
- ✓ Communicate your thought process.
- ✓ Identify entities, attributes and behaviors.
- ✓ Apply SOLID principles.
- ✓ Draw diagrams to explain your design.
- ✓ Keep the design extensible and maintainable.
- ✓ Review and refine the design.



Remember

Good design is simple, flexible and easy to extend.

OOP CONCEPTS

≈ The Foundation of LLD ≈

Page 4 of 10

? What is OOP?

Object Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data (attributes) and code (methods).

Benefits of OOP

- ✓ Modularity
- ✓ Reusability
- ✓ Maintainability
- ✓ Scalability
- ✓ Flexibility
- ✓ Real-world modeling

• The 4 Pillars of OOP •

1. Encapsulation

Bundling data (attributes) and methods that operate on the data into a single unit (class). Restrict direct access to some components.
Example: Bank Account (private balance, deposit()/withdraw())

2. Abstraction

Hiding unnecessary details and showing only essential features to the user.
Example: Shape (abstract draw())
→ Circle, Rectangle

3. Inheritance

Creating a new class from an existing class. The child inherits properties and behaviors of the parent.
Example: Vehicle (parent) → Car (child)

4. Polymorphism

The ability of an object to take many forms. Same interface, different implementations.
Example: Payment interface → UPI/Card/Wallet Payment

Class vs Object

- ★ **Class** — Blueprint or template for creating objects.
- ★ **Object** — Instance of a class.

```
class Student { { name, rollNo } → creates →
s1: Student(name="Abhi", rollNo=101), s2:
Student(name="Rahul", rollNo=102)
```

Relationships Between Classes

- ★ Association (uses) — $A \rightarrow B$
- ★ Aggregation (has-a) — $A \diamond \rightarrow B$
- ★ Composition (strong has-a) — $A \blacklozenge \rightarrow B$
- ★ Inheritance (is-a) — $A \rightarrow B$
- ★ Dependency (depends on) — $A \dashrightarrow B$

💡 OOP Principle

Treat objects as real-world entities. Focus on **what** they do, not **how** they do it internally.

Access Modifiers (Visibility)

Modifier	Same Class	Same Package	Sub Class (Different Package)	Other Package (Non-Sub Class)	Description
public	✓	✓	✓	✓	Accessible everywhere.
protected	✓	✓	✓	✗	Accessible in same package and subclasses.
default (package-private)	✓	✓	✗	✗	Accessible in the same package only.
private	✓	✗	✗	✗	Accessible within the class only.

Best Practices

- ✓ Follow SOLID Principles.
- ✓ Keep classes small and focused.
- ✓ Prefer composition over inheritance.
- ✓ Program to interfaces, not implementations.

📖 Remember

Good OOP design makes code easy to understand, change and extend.

SOLID PRINCIPLES

Page 5 of 10

≈ Build Strong, Maintainable Designs ≈

What is SOLID?

SOLID is a set of five design principles that help us create code that is easier to understand, maintain and extend.

Why it Matters?

- ✓ Reduces coupling between classes
- ✓ Improves code readability
- ✓ Enhances reusability and testability
- ✓ Makes the system easier to change and extend

The 5 SOLID Principles

1. Single Responsibility (SRP)

A class should have only one reason to change — one responsibility.
Ex: split Invoice.calculateTotal() and InvoicePrinter.printInvoice() into separate classes.

2. Open/Closed (OCP)

Entities should be open for extension but closed for modification.
Ex: Shape interface + draw() → add new shapes without changing existing code.

3. Liskov Substitution (LSP)

Subclasses should be replaceable for their superclass without breaking the app.
Ex: Penguin (can't fly) should NOT extend Bird if Bird.fly() is expected to work.

4. Interface Segregation (ISP)

Don't force a class to implement interfaces it does not use.
Ex: split Worker into work()/eat()/sleep() so Robot only implements work().

5. Dependency Inversion (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.
Ex: OrderService depends on PaymentGateway interface, not Razorpay/PayPal directly.

Each class does one job. We can add new shapes without changing existing code. Keep interfaces small and specific. Depend on abstraction (interface), not on concrete classes.

Common OOP Concepts in LLD



Class

Blueprint for creating objects.
class Car {}



Object

Instance of a class.
c1 = new Car();



Attribute

Properties of a class.
String name;



Method

Functions or behaviors.
void start()



Constructor

Special method to initialize objects.
Car() {}



Encapsulation

Binding data and methods together, restricting direct access.
private String name;



Abstraction

Showing only essential details, hiding implementation.
interface Shape {}



Inheritance

Deriving new class from existing class.
class Circle extends Shape {}



Polymorphism

One interface, many implementations.
Shape s = new Circle(); s.draw();

When to Use OOP in LLD?

- ✓ When the system has many related entities.
- ✓ When the system is expected to grow and change.
- ✓ When we need reusability and extensibility.
- ✓ When modeling real-world scenarios.



Remember: OOP is a tool, not a rule.

UML DIAGRAMS

≈ Visualize, Design & Communicate ≈

What is UML?

UML (Unified Modeling Language) is a standard visual language used to model software systems. It helps in designing, visualizing and documenting systems.

Why UML?

- ✓ Provides a common way to visualize systems
- ✓ Improves communication among team members
- ✓ Helps in understanding complex systems
- ✓ Acts as a blueprint for developers

💡 Tip: You don't need to use all diagrams every time. Use the ones that add the most value.

• Types of UML Diagrams •

1. Class Diagram

Shows the static structure of the system — classes, attributes, methods and relationships.
Use for: Modeling the structure of classes and their relationships.

2. Sequence Diagram

Shows object interactions in a time sequence.
Use for: Understanding how objects interact with each other over time.

3. Activity Diagram

Shows the flow of activities or business processes.
Use for: Modeling workflows and business processes.

4. State Diagram

Shows different states of an object and transitions between them.
Use for: Modeling state transitions of objects.

5. Use Case Diagram

Shows system functionality from the user's perspective.
Use for: Capturing requirements and system scope.

6. Component Diagram

Shows the components of the system and their dependencies.
Use for: Modeling system components and their dependencies.

7. Deployment Diagram

Shows the physical deployment of artifacts on hardware nodes.
Use for: Modeling deployment architecture.

8. Package Diagram

Shows how packages are organized and depend on each other.
Use for: Organizing model elements into packages.

9. Communication Diagram

Shows object interactions and the links between them.
Use for: Alternative to sequence diagram (focus on links).

10. Object Diagram

Shows instances of classes at a specific point in time.
Use for: Showing snapshot of objects and their values.

Relationship Notations (in Class Diagram)

- ★ Association (uses) — A → B
- ★ Aggregation (has-a) — A ◇→ B
- ★ Composition (strong has-a) — A ◆→ B
- ★ Inheritance (is-a) — A —▷ B
- ★ Dependency (depends on) — A -.-> B

Visibility

+ Public → Accessible everywhere
- Private → Accessible within class
Protected → Accessible in class & subclasses
~ Package → Accessible within package

Multiplicity

1 → Exactly one
0..1 → Zero or one
0.* → Zero or more
1.* → One or more
m..n → Between m and n
n → Exactly n

Example: Order (1) ◆has-a→ (1..*) OrderItem
class Order { id, date, calculateTotal() } — id, quantity, price

When To Use Which Diagram?

Diagram	Use When...
Class Diagram	You want to design the structure of classes & dependencies.
Sequence Diagram	You want to show interactions & order of messages between objects.
Component Diagram	You want to show components & dependencies at system level.
Deployment Diagram	You want to show physical deployment of the system.

📌 Remember: Diagrams are not documents. They exist to help you think and communicate design clearly.

DESIGN PATTERNS — DEEP DIVE (1/2)

Page 7 of 10

≈ Solutions to Common Problems ≈



What are Design Patterns?

Design Patterns are general, reusable solutions to commonly occurring problems in software design. They are not finished designs — they are templates or blueprints that you can adapt to solve problems in your code.

Types of Design Patterns (Gang of Four)

- **Creational** — Object creation mechanisms.
- **Structural** — Object composition & relationships.
- **Behavioral** — Communication between objects.

• Patterns 1 – 5 •

1. Singleton

Intent: Ensure a class has only one instance and provide a global point of access to it.

Structure: Client → Singleton.getInstance()
Example: Logger, Config Manager, DB Connection Pool

Real-World: Only one President in a country.

2. Factory

Intent: Define an interface for creating an object, but let subclasses decide which to instantiate.

Structure: Client → Factory.create(type) → ProductA / ProductB

Example: Payment Gateway, Notification, Vehicle Factory
Real-World: A factory produces different products based on requirement.

3. Strategy

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable.

Structure: Context(strategy) → Strategy interface → StrategyA / StrategyB

Example: Payment Methods, Sorting Algorithms, Pricing Engine
Real-World: Different routes to reach the same destination.

4. Observer

Intent: Define a one-to-many dependency so that when one object changes, all its dependents are notified.

Structure: Subject(attach/notify) → Observer1 / Observer2 / ObserverN

Example: Notification Service, Event Bus, Stock Updates
Real-World: A news channel and its subscribers.

5. Builder

Intent: Separate construction of a complex object from its representation.

Structure: Director → Builder(buildPartA/buildPartB/getResult) → Product

Example: User Builder, HTTP Request, SQL Query Builder
Real-World: Building a house step by step.

Situation	Use This Pattern
Need only one instance	Singleton
Create objects without exposing instantiation logic	Factory
Need multiple algorithms	Strategy
One-to-many dependency (notification)	Observer
Complex object construction (step by step)	Builder



Best Practices

Prefer composition over inheritance · Program to interfaces, not implementations · Keep classes small and focused · Follow SOLID principles · Choose the right pattern, not any pattern · Don't overuse patterns.

DESIGN PATTERNS — DEEP DIVE (2/2)

Page 8 of 10

≈ Solutions to Common Problems ≈

• Patterns 6 – 10 •

6. Decorator

Intent: Attach additional responsibilities to an object dynamically, without altering its structure.

Structure: Component ← ConcreteComponent · Decorator(component) wraps it

Example: Coffee Machine, Pizza Toppings, Logging, Compression

Real-World: Adding toppings to a pizza.

7. Adapter

Intent: Convert the interface of a class into another interface clients expect.

Structure: Client → Target interface ← Adapter → Adaptee (legacy class)

Example: Third-party APIs, Payment Providers, Legacy Systems

Real-World: Using a power adapter to fit a plug into a socket.

8. Command

Intent: Encapsulate a request as an object, thereby letting you parameterize clients with different requests.

Structure: Client → Invoker → Receiver(action) → Command(execute)

Example: Undo/Redo, Task Scheduler, Queue Processing

Real-World: Remote control for a TV.

9. State

Intent: Allow an object to alter its behavior when its internal state changes.

Structure: Context(setState/request) → State interface → ConcreteStateA / ConcreteStateB

Example: ATM Machine, Elevator, Vending Machine

Real-World: Different states of a fan (Off, Low, Medium, High).

10. Chain of Responsibility

Intent: Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request.

Structure: Client → Handler1 → Handler2 → ... → HandlerN

Example: Authentication, Logging, Approval Workflow

Real-World: Escalation process in customer support.

Situation	Use This Pattern
Add responsibilities dynamically	Decorator
Incompatible interfaces	Adapter
Encapsulate a request as an object (undo/redo)	Command
Behavior changes with internal state	State
Pass request along a chain	Chain of Responsibility

★ Remember

Design patterns are not magic. They are experience from other developers. Use the right pattern at the right time — don't force it.

Pattern Combination Examples

- ★ MVC → Strategy, Observer
- ★ Decorator → Composite
- ★ Adapter → Facade
- ★ Abstract Factory → Builder
- ★ Command → Memento
- ★ Logging System → Decorator + Singleton

CREATIONAL PATTERNS

Page 9 of 10

≈ How Objects Are Created ≈

Beyond Singleton, Factory and Builder — a few more creational patterns worth knowing.

Prototype

Create new objects by copying existing instances.
Document doc1 = new Document(); Document
doc2 = doc1.clone();
Example: Document Clone, Image Loading

Abstract Factory

Provide an interface for creating families of related or dependent objects.
UIFactory → WindowsFactory / MacFactory
(createButton, createCheckbox)
Example: UI Components (cross-platform widgets)

Object Pool

Reuse objects that are expensive to create. Pool maintains Connection objects; client borrows/returns.
Example: DB Connection Pool

Lazy Loading

Delay object creation until it is needed.
Client → Proxy(LazyLoader) → Real Object (loaded on demand)
Example: Image Loading, Vehicle proxy

Factory Method

Define an interface for creating an object, let subclasses decide.
VehicleFactory(createVehicle) → CarFactory / BikeFactory
Example: Vehicle creation (Car, Bike)

Monostate

Similar to Singleton but allows multiple instances with shared state.
Object1, Object2 → share the same underlying (Common Data)
Example: App Settings (shared config)

Singleton vs Monostate vs Static

Feature	Singleton	Monostate	Static
Instances	Only one	Multiple allowed	No instance
State	Unique to instance	Shared across all	Shared globally
Memory	Single object	Multiple objects	No object needed
Flexibility	★★★☆☆	★★★★☆	★☆☆☆☆
When to use	Need only one object with shared state	Multiple instances with shared state & constants	Utility functions

When to Use Which Pattern?

Singleton	→ When you need only one object creation.
Factory	→ When object creation logic is complex.
Builder	→ When object has many parts/steps.
Prototype	→ When object creation is costly.
Abstract Factory	→ When you need families of related objects.
Object Pool	→ When resource is heavy and not always needed.
Factory Method	→ When subclasses decide the object type.
Monostate	→ When multiple instances share the same state.

Pro Tip

Don't just mug up the patterns. Understand the PROBLEM they solve and how they add VALUE to the design.

• Design Patterns in Real-World Systems •

E-Commerce (Flipkart/Amazon)

Factory → Payment Gateway ·
Strategy → Discounts & Offers ·
Observer → Order Status Updates ·
Builder → Order Creation

Ride Booking App (Uber/Ola)

Strategy → Fare Calculation ·
Observer → Driver Location Updates ·
Factory → Vehicle Type (Cab, Auto, Bike)

Food Delivery App (Swiggy/Zomato)

Observer → Chat Notifications ·
Singleton → Chat Manager/Config ·
Factory → Message Type (Text, Image, File) ·
Decorator → Add-ons (Extra Cheese, etc.)

Banking System

Singleton → DB Connection ·
Strategy → Loan Calculation ·
Chain of Responsibility →
Approval Process ·
Adapter → Third-party Payment Gateway

Quick Memory Tricks

Singleton → One is the Only One.
Factory → Factory produces products.
Builder → Build step by step.

Interview Tip

Always explain WHY you chose a pattern, not just WHAT it does.

Golden Rule

There is no "best" pattern — only the right pattern for your problem.

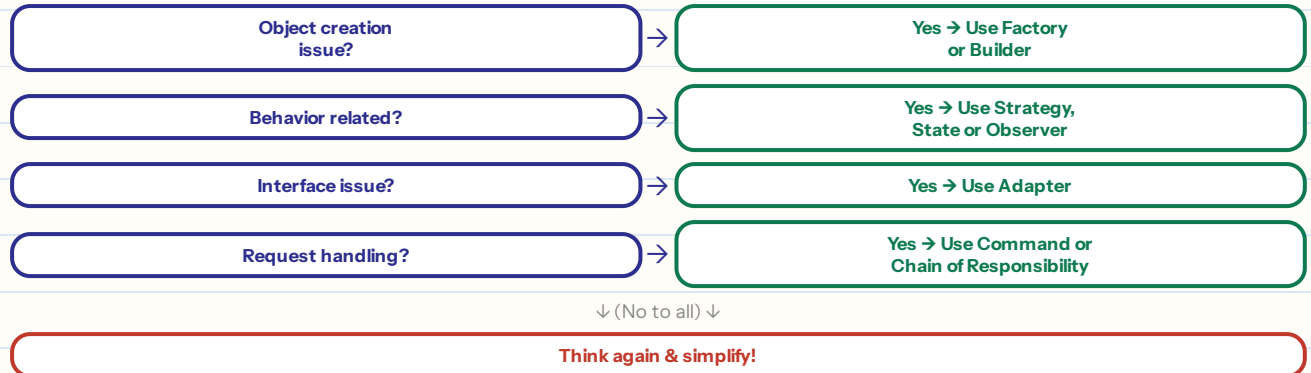
PATTERN SELECTION GUIDE

≈ Cheat Sheet & Final Recap ≈

Page 10 of 10

Pattern Selection Flow

What is the problem?



When to Use Which Pattern?

Problem / Situation	Use This Pattern
Need only one instance	Singleton
Create objects without exposing instantiation logic	Factory
Need multiple algorithms	Strategy
One-to-many dependency (notification)	Observer
Complex object construction (step by step)	Builder
Add responsibilities dynamically	Decorator
Incompatible interfaces	Adapter
Encapsulate a request	Command
Behavior changes with state	State
Pass request along a chain	Chain of Responsibility

Best Practices (Recap)

- ✓ Don't overuse patterns.
- ✓ Start simple, add patterns when needed.
- ✓ Prefer composition over inheritance.
- ✓ Program to interfaces, not implementations.
- ✓ Keep classes small and focused.
- ✓ Follow SOLID principles.
- ✓ Choose the right pattern for the right job.
- ✓ Understand the trade-offs.
- ✓ Keep it simple, stupid (KISS).
- ✓ Refactor, improve, repeat.

✚ Pattern Combination Cheatsheet

- ★ MVC → Strategy + Observer
- ★ Logging System → Decorator + Singleton
- ★ Payment Gateway → Adapter + Factory
- ★ Notification System → Observer + Command

🔍 Pattern Recognition Tips

- ◆ Identify the problem first.
- ◆ Think about flexibility and change.
- ◆ Look at objects, relationships and responsibilities.

🎯 Final Reminder

Patterns are tools in your toolbox. Use the right tool at the right time. Code is read more often than it is written — make it clean!

✓ Key Takeaways

Design Patterns are proven solutions to recurring problems. They make code reusable, maintainable and scalable. They are not frameworks, but blueprints you can adapt.

★ Remember

A good developer knows how to code. A great developer knows when to use a pattern. Practice patterns, build better systems.

Notes compiled from a 10-part LLD Design Patterns series — @abhi_techhub